

SnapSight — Special Prompting Structure

A copy-paste prompt book to build the project step-by-step with AI coding assistants.

Format: Role → Objective → Context → Instructions → Notes

How to use: Ek time par sirf ek prompt use karo. AI se pehle planning, phir one-module implementation, phir testing/debugging karwao. Whole project ek hi prompt mein generate mat karwana.

PROJECT: SnapSight — privacy-first multimodal AI assistant for Snapdragon-powered HP PCs.

MVP: Screen Understanding + Voice Understanding + Camera Understanding + Context Mode.

IMPORTANT: Qualcomm AI Hub/model/runtime/NPU support ko current official documentation se verify karna hai. AI ko unverified Qualcomm API invent karne nahi dena.

1. MASTER PROJECT PROMPT

****Role:****

You are a senior AI/edge-AI software architect and mentor helping me build a real hackathon project.

****Objective:****

Help me build SnapSight, a privacy-first multimodal AI assistant designed for Snapdragon-powered HP PCs.

****Context:****

The MVP has four major capabilities:

1. Understand Screen
2. Understand Camera
3. Understand Voice
4. Context Mode that combines available modalities.

Tech preference:

- Python
- FastAPI
- HTML/CSS/JavaScript
- OpenCV
- Qualcomm AI Hub compatible models
- Windows on Snapdragon

****Instructions:****

*****Instruction 1:*****

First understand the complete project and propose a modular architecture.

*****Instruction 2:*****

Do NOT write the whole project at once. Break it into small modules that I can run and test independently.

*****Instruction 3:*****

For every module give: files, dependencies, setup commands, code, test command, expected output and common errors.

*****Instruction 4:*****

Do not invent Qualcomm APIs, model names, runtime support or NPU execution. If something must be verified, clearly mark it as VERIFY.

*****Instruction 5:*****

Use simple Hinglish explanations because I am a college student.

****Notes:****

- Build an MVP first.
- Keep code beginner-readable.
- Never put API keys in code.
- Never use fake benchmark numbers.
- Do not move to the next module until the current module works.

2. ARCHITECTURE PROMPT

****Role:****

You are a senior system architect specializing in edge AI and multimodal applications.

****Objective:****

Design the complete technical architecture for SnapSight before any code is written.

****Context:****

SnapSight receives:

- screen/screenshot input
- camera frame input
- microphone/audio input

It should process these locally where supported and produce a useful context-aware response.

****Instructions:****

*****Instruction 1:*****

Create the high-level architecture.

Instruction 2:

Create the data flow for Screen → AI → Response.

Instruction 3:

Create the data flow for Camera → AI → Response.

Instruction 4:

Create the data flow for Mic → Speech Model → Text → Response.

Instruction 5:

Create the final Context Engine flow combining multiple inputs.

Instruction 6:

Suggest a clean Python/FastAPI folder structure.

Notes:

- Do not choose unverified Qualcomm components.
- Clearly separate UI, API, model inference and context logic.
- Explain every component in simple Hinglish.

3. MODEL RESEARCH PROMPT

Role:

You are an edge-AI model deployment researcher.

Objective:

Help me select models for SnapSight that are realistically deployable on my target Snapdragon-powered HP PC.

Context:

I need:

1. Speech-to-text
2. Vision/image understanding
3. Optional OCR
4. Optional small language model

Instructions:

Instruction 1:

For each candidate model, identify its exact official model name.

Instruction 2:

Identify Qualcomm AI Hub availability if applicable.

Instruction 3:

Identify supported target devices, runtime and input/output format.

Instruction 4:

Identify whether NPU execution is officially supported or needs verification.

Instruction 5:

Give me a final shortlist based on compatibility and implementation complexity, but do not invent missing information.

Notes:

- Use current official sources when checking Qualcomm compatibility.
- Do not claim a model is NPU-optimized unless verified.
- If a model is not verified for my target, mark it VERIFY.

4. SCREEN UNDERSTANDING PROMPT

****Role:****

You are a Python computer-vision engineer.

****Objective:****

Build only the Screen Understanding module for SnapSight.

****Context:****

The user clicks "Understand Screen". The application captures a screenshot, processes it, sends it through the verified vision/OCR pipeline, and returns useful context.

****Instructions:****

*****Instruction 1:*****

Create only the files required for the screen module.

*****Instruction 2:*****

Implement screenshot capture for Windows.

*****Instruction 3:*****

Add preprocessing if required by the selected model.

*****Instruction 4:*****

Connect the screenshot to the verified model pipeline.

*****Instruction 5:*****

Return a clean structured result such as screen_text, visual_context and status.

*****Instruction 6:*****

Add error handling for permissions, missing model files and invalid input.

*****Instruction 7:*****

Give exact installation and test commands.

****Notes:****

- Do not implement camera, microphone or Context Mode yet.
- Do not invent model APIs.
- Use a temporary in-memory image where possible.
- Explain every important line in Hinglish.

5. VOICE / WHISPER PROMPT

****Role:****

You are a speech-AI engineer experienced with local inference.

****Objective:****

Build the Voice Understanding module for SnapSight.

****Context:****

User clicks "Understand Voice", records approximately 5-10 seconds, speech is converted to text, and the transcript becomes a context input.

****Instructions:****

*****Instruction 1:*****

Implement microphone recording on Windows.

*****Instruction 2:*****

Preprocess audio exactly as required by the selected verified model.

*****Instruction 3:*****

Integrate the verified Whisper-family model/runtime.

*****Instruction 4:*****

Return transcript, processing status and error information.

*****Instruction 5:*****

Show how to test with a short sentence.

****Notes:****

- Do not assume a Qualcomm runtime or NPU path without verification.
- Keep recording length configurable.
- Do not store recordings permanently unless required.
- Explain the model input/output clearly.

6. CAMERA UNDERSTANDING PROMPT

****Role:****

You are a computer-vision engineer building an edge-AI application.

****Objective:****

Build the Camera Understanding module for SnapSight.

****Context:****

The user clicks "Understand Camera". The webcam opens, one frame is captured, and the selected vision model returns a description or structured visual information.

****Instructions:****

*****Instruction 1:****

Implement webcam initialization.

*****Instruction 2:****

Capture one frame first; do not build continuous 30-FPS inference.

*****Instruction 3:****

Preprocess the frame according to the verified model.

*****Instruction 4:****

Run inference and return the result.

*****Instruction 5:****

Add handling for camera permission and camera-not-found errors.

****Notes:****

- Single-frame MVP first.
- Continuous mode is optional later.
- Do not claim real-time performance without measuring it.

7. CONTEXT ENGINE PROMPT

****Role:****

You are a multimodal AI application engineer.

****Objective:****

Build SnapSight's Context Engine.

****Context:****

Possible inputs:

- screen_context
- camera_context
- voice_query

The engine should use only available information and generate a concise useful response.

****Instructions:****

*****Instruction 1:*****

Define a clean Context object/schema.

*****Instruction 2:*****

Handle missing inputs safely.

*****Instruction 3:*****

Combine at least two modalities when available.

*****Instruction 4:*****

Create a response function that does not invent facts.

*****Instruction 5:*****

Add logging showing which modalities were used.

****Notes:****

- Start with deterministic/rule-based combination if necessary.
- A local small LLM may be added later.
- Never hallucinate information not present in the supplied context.

8. FRONTEND/UI PROMPT

****Role:****

You are a UI/UX engineer building a polished hackathon demo.

****Objective:****

Create a clean SnapSight dashboard that makes the multimodal workflow easy to understand.

****Context:****

The UI needs four actions:

- Understand Screen
- Understand Camera
- Understand Voice
- Context Mode

It should also show current processing state and AI response.

****Instructions:****

*****Instruction 1:*****

Create a responsive dashboard using HTML/CSS/JavaScript.

*****Instruction 2:*****

Show clear loading, success and error states.

*****Instruction 3:*****

Show which input modality is currently active.

*****Instruction 4:*****

Add a visible "Local AI / On-device processing" status only where technically accurate.

Instruction 5:

Keep the design professional and simple.

Notes:

- Avoid unnecessary animations.
- The demo should be readable in a screen recording.
- Do not claim local inference for a cloud API.

9. FASTAPI INTEGRATION PROMPT

Role:

You are a backend engineer specializing in FastAPI.

Objective:

Connect SnapSight's frontend to the independent AI modules.

Context:

Existing modules:

- screen.py
- speech.py
- vision.py
- context.py

Instructions:

Instruction 1:

Create clean API endpoints for each module.

Instruction 2:

Validate inputs.

Instruction 3:

Return structured JSON responses.

Instruction 4:

Add CORS only as required for the local frontend.

Instruction 5:

Add clear exception handling and logging.

Notes:

- Do not put model logic directly inside route functions.
- Keep endpoints small.
- Provide curl/test examples.
- Do not expose secrets.

10. DEBUGGING PROMPT

Role:

You are a senior debugging engineer.

Objective:

Find and fix the smallest possible cause of my SnapSight error.

Context:

Project:

SnapSight

Module:

[WRITE MODULE]

Environment:

Windows:

Python:

Model:

Runtime:
Target Snapdragon:

Complete error:
[PASTE ERROR]

****Instructions:****

*****Instruction 1:*****

Identify the root cause.

*****Instruction 2:*****

Point to the exact file/line if possible.

*****Instruction 3:*****

Explain why it happens in simple Hinglish.

*****Instruction 4:*****

Give the smallest safe fix.

*****Instruction 5:*****

Give one verification command.

****Notes:****

- Do not rewrite the entire project.
- Do not invent missing APIs.
- Ask me for the relevant file only if necessary.

11. QUALCOMM AI HUB VERIFICATION PROMPT

****Role:****

You are a Qualcomm AI deployment engineer.

****Objective:****

Verify whether my selected model can be deployed correctly on my target Snapdragon-powered HP PC.

****Context:****

Target device:

[EXACT HP + SNAPDRAGON MODEL]

OS:

Windows 11

Model:

[MODEL]

****Instructions:****

*****Instruction 1:*****

Verify the official model listing.

*****Instruction 2:*****

Verify target-device support.

*****Instruction 3:*****

Verify runtime.

*****Instruction 4:*****

Verify whether NPU execution is supported.

*****Instruction 5:*****

Identify the correct deployment/evaluation workflow.

*****Instruction 6:*****

Tell me what evidence I should capture for the hackathon submission.

****Notes:****

- Never assume an API exists.
- If something cannot be verified, write VERIFY.
- Do not create fake benchmark numbers.
- Prefer official Qualcomm documentation.

12. PERFORMANCE/BENCHMARK PROMPT

****Role:****

You are an edge-AI benchmarking engineer.

****Objective:****

Create a fair benchmark plan for SnapSight on my Snapdragon PC.

****Context:****

I need evidence for:

- model inference time
- memory usage if measurable
- compute target
- repeated-run consistency

****Instructions:****

*****Instruction 1:*****

Define the test conditions.

*****Instruction 2:*****

Tell me how many runs to perform.

*****Instruction 3:*****

Tell me what values to record.

Instruction 4:

Show how to calculate average and median latency.

Instruction 5:

Tell me how to document the benchmark honestly.

Notes:

- Do not compare against another platform unless I actually run the comparison.
- Do not use synthetic numbers.
- Clearly state model version, input size and runtime.

13. INTEGRATION TEST PROMPT

Role:

You are a QA engineer for a multimodal AI application.

Objective:

Test SnapSight end-to-end before the demo.

Context:

Features:

Screen + Camera + Voice + Context Mode.

Instructions:

Instruction 1:

Create a test matrix.

Instruction 2:

Test each module independently.

Instruction 3:

Test combinations:

screen + voice

camera + voice

screen + camera

screen + camera + voice

Instruction 4:

Test failure cases:

camera unavailable

mic unavailable

model missing

invalid image

empty transcript

slow inference

Instruction 5:

Give pass/fail criteria.

Notes:

- Record bugs before final demo.
- Fix crashes before adding new features.

14. GITHUB README PROMPT

Role:

You are a technical writer and open-source engineer.

Objective:

Write a professional README for SnapSight.

Context:

The README must make the project understandable to judges and developers.

****Instructions:****

Include:

1. Project overview
2. Problem
3. Solution
4. Features
5. Architecture
6. AI models
7. Qualcomm AI Hub integration
8. Snapdragon optimization
9. Verified compute/NPU evidence
10. Installation
11. Usage
12. Screenshots
13. Demo
14. Performance
15. Limitations
16. Future scope
17. License

****Notes:****

- Clearly separate verified facts from future plans.
- Never invent performance values.
- Keep technical claims reproducible.

15. DEMO SCRIPT PROMPT

****Role:****

You are a hackathon demo and presentation coach.

****Objective:****

Create a 2-3 minute live demo script for SnapSight.

****Context:****

The audience should quickly understand:

Problem → Solution → AI → Snapdragon → Result.

****Instructions:****

*****Instruction 1:*****

Write a 20-second problem introduction.

*****Instruction 2:*****

Introduce SnapSight in one sentence.

*****Instruction 3:*****

Create a live Screen demo.

*****Instruction 4:*****

Create a Voice demo.

*****Instruction 5:*****

Create a Camera demo.

*****Instruction 6:*****

Create a Context Mode wow moment.

*****Instruction 7:*****

Show actual Qualcomm/Snapdragon evidence.

*****Instruction 8:*****

End with the project's technical value.

****Notes:****

- Keep the demo reproducible.
- Have a fallback recording/screenshots.
- Never claim unsupported NPU/latency results.

16. FINAL SUBMISSION CHECK PROMPT

****Role:****

You are my final hackathon submission reviewer.

****Objective:****

Audit SnapSight before I submit it.

****Context:****

Evaluation areas include:

- Technical Implementation
- Application Use Case & Innovation
- Deployment & Accessibility
- Presentation & Documentation

****Instructions:****

*****Instruction 1:*****

Review my project evidence against each criterion.

*****Instruction 2:*****

Identify missing technical proof.

*****Instruction 3:*****

Identify unclear documentation.

Instruction 4:

Identify demo risks.

Instruction 5:

Identify claims that need verification.

Instruction 6:

Give me a final submission checklist.

Notes:

- Do not tell me that I will win.
- Focus on concrete improvements and missing evidence.
- Separate facts, assumptions and future work.

17. HOW TO USE THESE PROMPTS

Order	Prompt	Use when
1	Master Project	Start project
2	Architecture	Before coding
3	Model Research	Before choosing models
4	Screen	Build first AI module
5	Voice	Build speech module
6	Camera	Build vision module
7	Context Engine	Combine modules
8	Frontend	Build UI
9	FastAPI	Connect backend
10	Debugging	Whenever an error occurs
11	Qualcomm Verification	Before Snapdragon claims
12	Benchmark	After deployment
13	Integration Test	Before demo
14	README	After stable build
15	Demo Script	Before recording
16	Final Check	Before submission

Golden rule: AI ko coder ki tarah use karo, autopilot ki tarah nahi. Har generated module ko khud run karo, error samjho, test karo aur Git commit karo.

Project target: Working MVP first → Qualcomm verification → real benchmark evidence → polished demo → final submission.